

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота № 2

з дисципліни «Тестування та контроль якості (QA) вбудованих систем»

Тема: «НАЛАШТУВАННЯ МЕРЕЖНОГО ОТОЧЕННЯ ТА
ТЕСТУВАННЯ ПРОТОКОЛУ ARP»

Виконала:

студентка групи ІА-31

Кизим Єлизавета

Київ 2026

Тема: Налаштування мережного оточення та тестування протоколу ARP

Мета: Навчитися налаштовувати мережне оточення для тестування вбудованих систем та пристроїв IoT. Навчитися використовувати утиліту wireshark для аналізу трафіка в комп'ютерній мережі. Протестувати мережне оточення на канальному рівні моделі OSI.

Завдання:

Останні дві цифри номера залікової книжки - 11, тому $11 \% 3 = 2$.

Варіант: 2

Визначення варіанту: Останні дві цифри номеру залікової книжки розділити на три. Остача від ділення і буде номером варіанту.

0) Переконалися в отриманні MAC-адреси по відомій IP-адресі.

1) Переконалися в тому, що *arp*-таблиця оновлюється при отриманні *arp-reply*.

2) Додати статичний запис у *arp*-таблицю та після цього пропінгувати інший пристрій, при цьому відстежувати трафік у *wireshark*.

Під час роботи необхідно згідно з варіантом:

1) Переконалися в отриманні MAC-адреси по відомій IP-адресі.

2) Переконалися в тому, що *arp*-таблиця оновлюється при отриманні *arp-reply*.

3) Додати статичний запис у *arp*-таблицю та після цього пропінгувати інший пристрій, при цьому відстежувати трафік у *wireshark* та **описати тест-кейс до цього пункту.**

Теоретичні відомості

Протокол **ARP** (Address Resolution Protocol) працює на канальному рівні моделі OSI та призначений для визначення фізичної MAC-адреси пристрою за його відомою мережевою IPv4-адресою у межах одного сегмента мережі. Коли вузол має намір відправити пакет, але не знає MAC-адресу отримувача, він генерує широкомовний запит (ARP Request) на адресу ff:ff:ff:ff:ff:ff. Вузол із відповідною IP-адресою надсилає одноадресну відповідь (ARP Reply), після чого отримана зв'язка «IP-MAC» кешується в ARP-таблиці відправника для подальшого використання. Утиліта ping застосовує протокол мережевого рівня ICMP (Internet Control Message Protocol) для перевірки доступності віддалених вузлів,

відправляючи повідомлення ICMP Echo Request та очікуючи на ICMP Echo Reply. Для успішної передачі ICMP-пакетів у локальній мережі (Ethernet-сегменті) відправнику необхідна MAC-адреса наступного вузла, яка може бути отримана динамічно через протокол ARP або заздалегідь прописана як статичний запис у таблиці.

Хід роботи

1. Інсталяція Wireshark

```
kyzym_y_k@Ubuntu:~$ sudo apt-get update
Hit:1 http://ie.archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://ie.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://ie.archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Reading package lists... Done
E: Release file for http://ie.archive.ubuntu.com/ubuntu/dists/resolute-updates/InRelease is not valid yet (
invalid for another 16d 17h 34min 31s). Updates for this repository will not be applied.
E: Release file for http://ie.archive.ubuntu.com/ubuntu/dists/resolute-backports/InRelease is not valid yet
(invalid for another 16d 6h 2min 42s). Updates for this repository will not be applied.
E: Release file for http://security.ubuntu.com/ubuntu/dists/resolute-security/InRelease is not valid yet (i
nvalid for another 16d 21h 43min 46s). Updates for this repository will not be applied.
kyzym_y_k@Ubuntu:~$
```

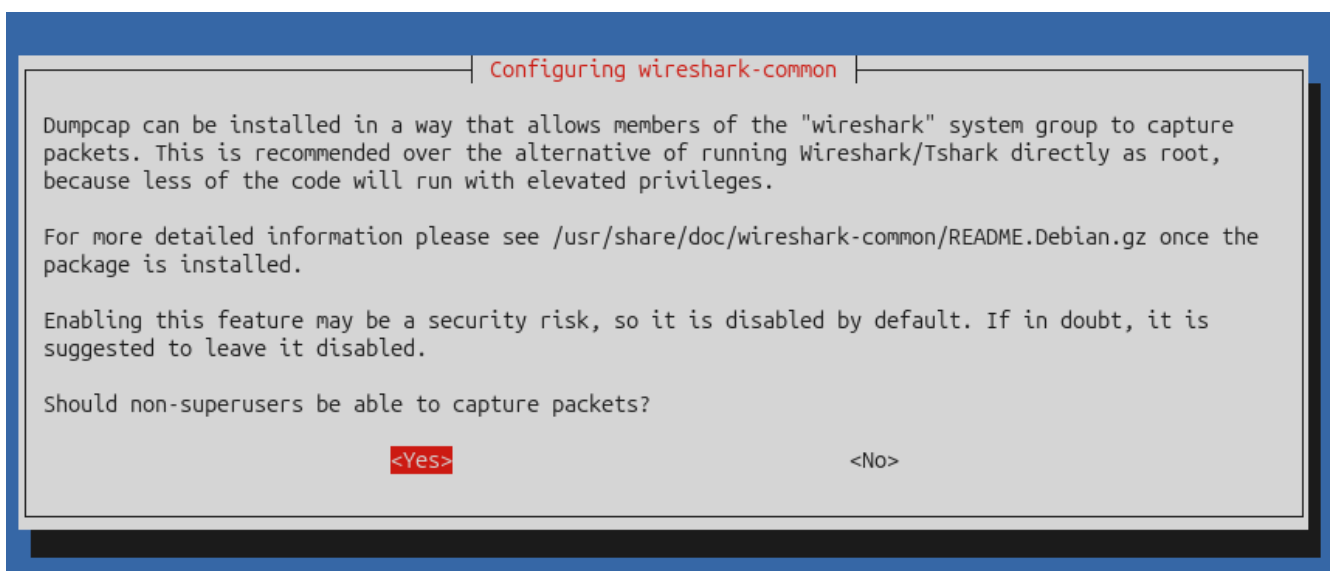
Введена команда **sudo apt-get update** ініціювала процес оновлення списків доступних пакетів з налаштованих репозиторіїв системи.

```
kyzym_y_k@Ubuntu:~$ sudo add-apt-repository ppa:wireshark-dev/stable
PPA publishes dbgsym, you may need to include 'main/debug' component
Repository: 'Types: deb
URIs: https://ppa.launchpadcontent.net/wireshark-dev/stable/ubuntu/
Suites: resolute
Components: main
'
Description:
Latest stable Wireshark releases back-ported from Debian package versions.
```

За допомогою команди **sudo add-apt-repository ppa:wireshark-dev/stable** до системи було успішно додано офіційний PPA-репозиторій розробників. Це дозволяє завантажити та інстальувати найновішу стабільну версію аналізатора мережевих протоколів Wireshark.

```
kyzym_y_k@Ubuntu:~$ sudo apt-get install wireshark
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Solving dependencies... Done
The following additional packages will be installed:
  libaacso libavcodec62 libavformat62 libavutil60 libb2-1 libbcg729-0 libbdplus0 libbluray3
  libchromaprint1 libcbjso1 libcodec2-1.2 libdav1d7 libdouble-conversion3 libdvdnav4 libdvdread8t64
  libfalcosecurity0t64 libgme0 libgrpc++1.51t64 libgrpc29t64 libgsm1 libhwloc-plugins libhwloc15
  libjsoncpp26 libmbedcrypto16 libmd4c0 libminizip1t64 libnghttp3-9 libnorm1t64 libopencore-amrnb0
  libopengl0 libopenmpt0t64 libpcre2-16-0 libpgm-5.3-0t64 libprotoc32t64 libqt6core5compat6
  libqt6core6t64 libqt6dbus6 libqt6gui6 libqt6multimedia6 libqt6network6 libqt6opengl6
  libqt6printsupport6 libqt6qml6 libqt6qmlmeta6 libqt6qmlmodels6 libqt6qmlworkerscript6 libqt6quick6
  libqt6svg6 libqt6waylandclient6 libqt6waylandcompositor6 libqt6widgets6 libqt6wlshellintegration6
  librabbitmq4 libravie0.8 libre2-11 librist4 libshine3 libsmi2t64 libsoxr0 libspandsp2t64
  libst1.5-0 libst1.5-0 libst1.5-0 libst1.5-0 libst1.5-0 libst1.5-0 libst1.5-0 libst1.5-0 libst1.5-0
```

Введена команда **sudo apt-get install wireshark** успішно розпочала процес встановлення програми. Термінал відобразив перелік додаткових пакетів та бібліотек (залежностей), які будуть автоматично встановлені для забезпечення коректної роботи аналізатора мережі.

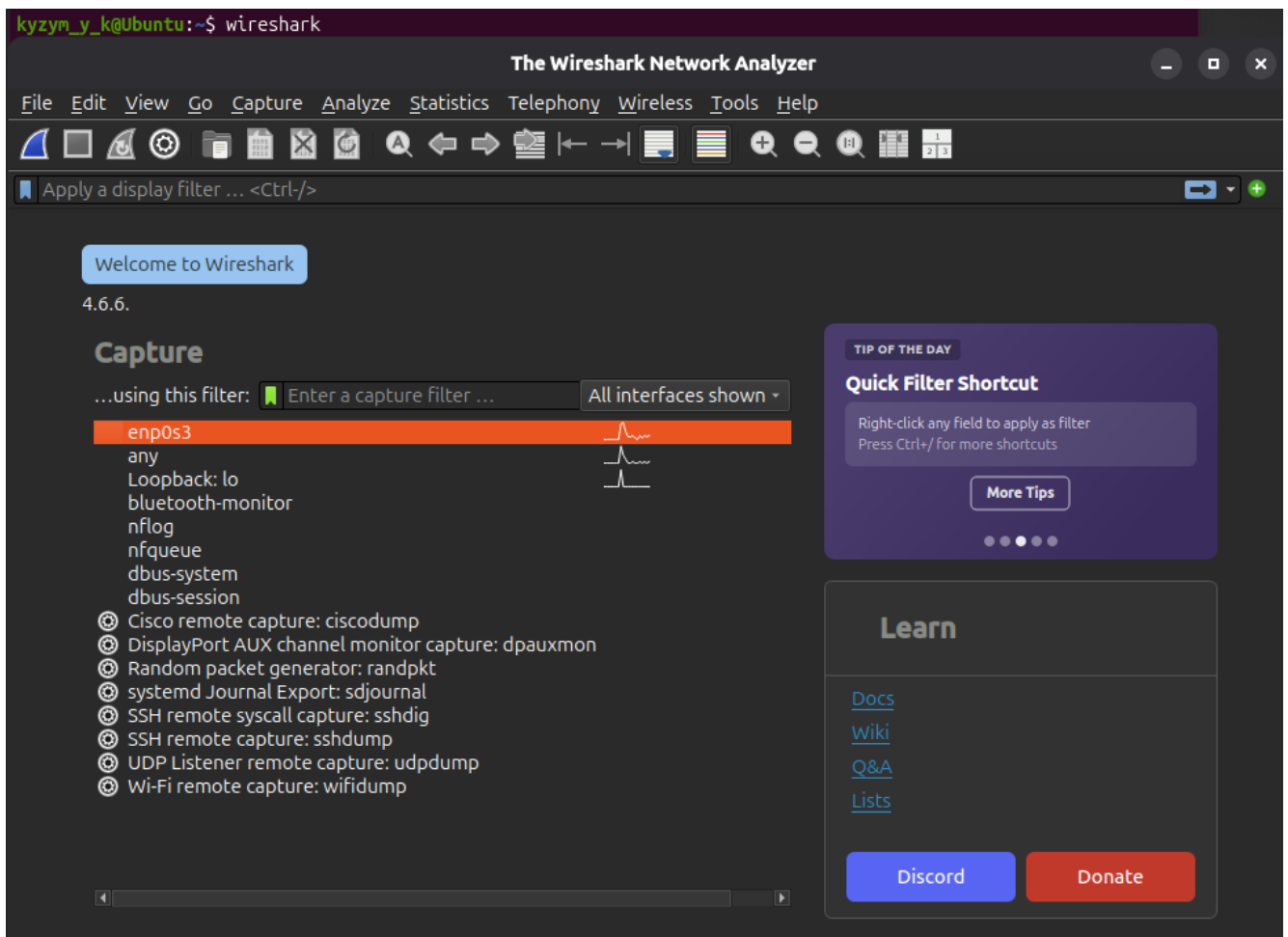


У процесі інсталяції на екрані з'явилося псевдографічне вікно конфігурації пакета **wireshark-common**. Було обрано опцію **<Yes>**, що дозволило змінити налаштування безпеки та надати право звичайним користувачам (не-суперкористувачам) перехоплювати мережеві пакети без постійного використання прав **root**.

```
kyzym_y_k@Ubuntu:~$ sudo adduser $USER wireshark
kyzym_y_k@Ubuntu:~$ cat /etc/group | grep wireshark
wireshark:x:115:kyzym_y_k
```

Оскільки на попередньому кроці було дозволено захоплення трафіку не-суперкористувачам, за допомогою команди **sudo adduser \$USER wireshark** поточний обліковий запис було успішно додано до спеціальної системної групи **wireshark**. Для перевірки успішності операції було використано команду читання системного файлу груп з фільтрацією виводу командою **grep**. Термінал вивів рядок **wireshark:x:115:kyzym_y_k**, що візуально підтверджує успішне додавання поточного користувача до вказаної групи.

2. Захоплення пакетів у Wireshark

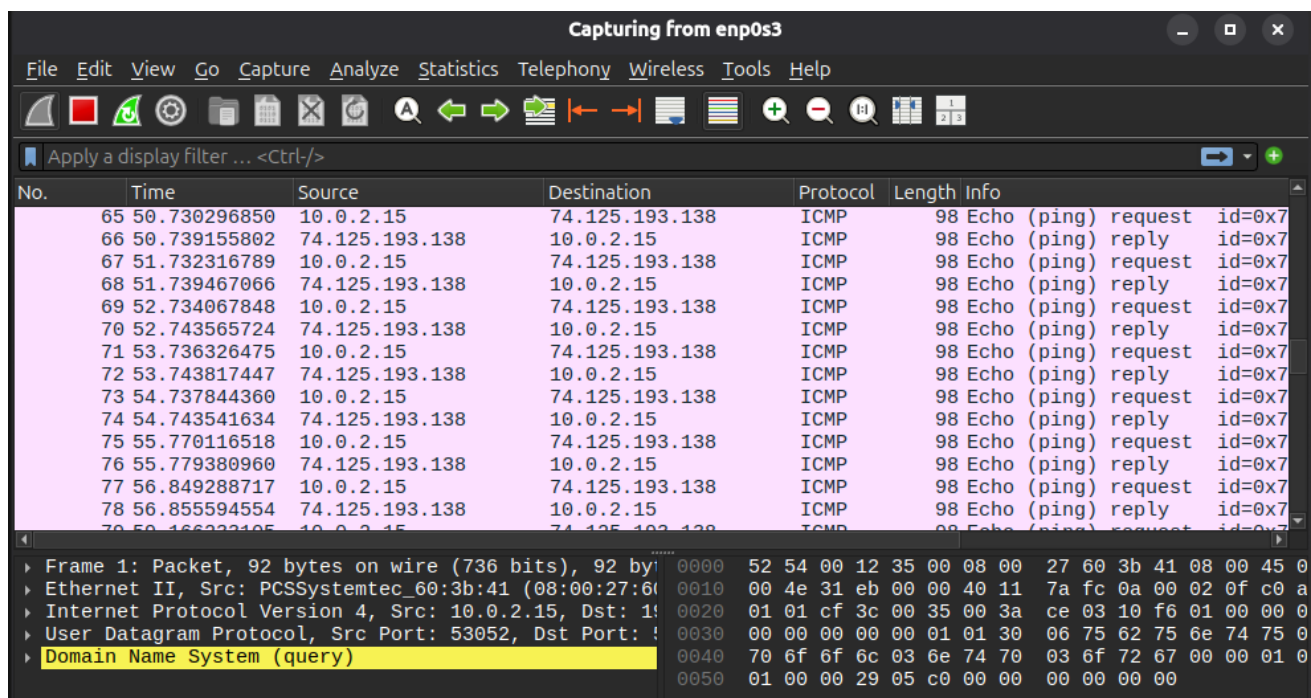


Після перезавантаження системи для застосування оновлених прав доступу, аналізатор мережевих протоколів було успішно запущено базовою командою **wireshark** без використання привілеїв суперкористувача. На скриншоті видно головне вікно програми, де в списку інтерфейсів для захоплення пакетів коректно відображаються локальні мережеві з'єднання (зокрема основний інтерфейс **enp0s3** та **Loopback: lo**). Наявність міні-графіків поруч із назвами інтерфейсів свідчить

про те, що програма успішно зчитує їхню поточну мережеву активність та повністю готова до роботи.

```
kyzym_y_k@Ubuntu:~$ ping google.com
PING google.com (74.125.193.138) 56(84) bytes of data.
64 bytes from ig-in-f138.1e100.net (74.125.193.138): icmp_seq=1 ttl=64 time=8.07 ms
64 bytes from ig-in-f138.1e100.net (74.125.193.138): icmp_seq=2 ttl=64 time=6.25 ms
64 bytes from ig-in-f138.1e100.net (74.125.193.138): icmp_seq=3 ttl=64 time=8.88 ms
64 bytes from ig-in-f138.1e100.net (74.125.193.138): icmp_seq=4 ttl=64 time=7.17 ms
64 bytes from ig-in-f138.1e100.net (74.125.193.138): icmp_seq=5 ttl=64 time=9.52 ms
64 bytes from ig-in-f138.1e100.net (74.125.193.138): icmp_seq=6 ttl=64 time=7.52 ms
64 bytes from ig-in-f138.1e100.net (74.125.193.138): icmp_seq=7 ttl=64 time=5.72 ms
64 bytes from ig-in-f138.1e100.net (74.125.193.138): icmp_seq=8 ttl=64 time=9.29 ms
```

У новому вікні термінала було введено команду **ping google.com** для перевірки доступності віддаленого хоста. Як видно з виводу, утиліта успішно відправила серію мережевих запитів та отримала відповіді від сервера з IP-адресою 74.125.193.138, що підтверджує наявність стабільного з'єднання.

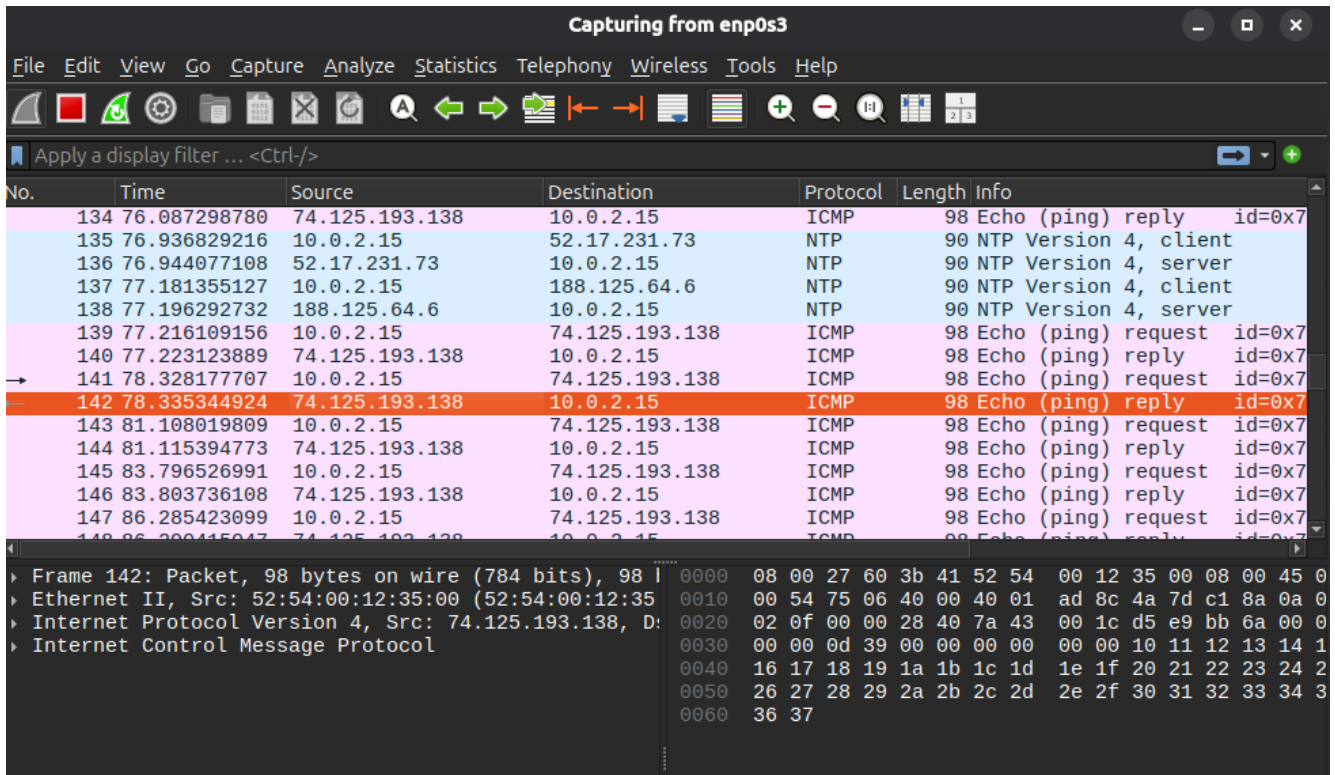


The image shows a Wireshark packet capture window titled "Capturing from enp0s3". The packet list on the left shows a series of ICMP Echo (ping) requests and replies. The selected packet (No. 65) is expanded in the packet details pane, showing the following structure:

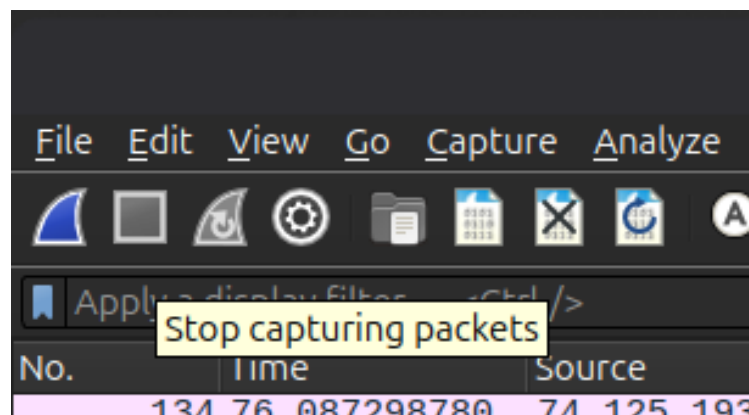
Frame 1: Packet, 92 bytes on wire (736 bits), 92 bytes captured (736 bits) on interface enp0s3
Ethernet II, Src: PCSSystemtec_60:3b:41 (08:00:27:60:3b:41), Dst: 10.0.2.15 (08:00:27:60:3b:41)
Internet Protocol Version 4, Src: 10.0.2.15, Dst: 74.125.193.138
User Datagram Protocol, Src Port: 53052, Dst Port: 80
Domain Name System (query)

Паралельно з виконанням команди **ping**, у програмі Wireshark було розпочато захоплення трафіку на обраному локальному інтерфейсі **enp0s3**. На скриншоті робочого вікна аналізатора чітко зафіксовано перехоплені пакети протоколу ICMP. У списку відображаються парні записи процесу: Echo (ping) request (запит від IP-адреси локальної машини 10.0.2.15 до сервера 74.125.193.138) та відповідні

ім Echo (ping) reply (відповідь від сервера до локальної машини). Це наочно демонструє базовий механізм перевірки мережевої доступності.

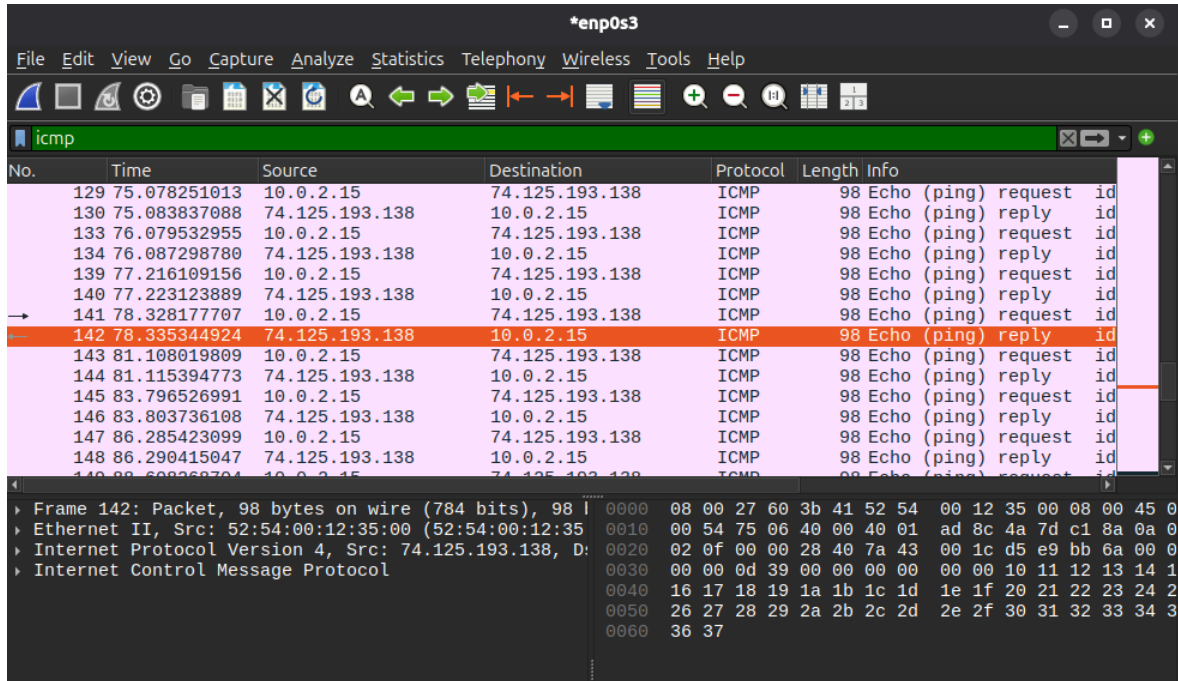


У вікні програми Wireshark зі списку перехопленого трафіку було виділено один конкретний пакет — відповідь сервера ICMP Echo (ping) reply. Після виділення, у середній панелі інтерфейсу стала доступною детальна інформація про структуру цього пакета, де можна розгорнути та дослідити кожен рівень його інкапсуляції (канальний рівень Ethernet II, мережевий рівень IPv4 та сам протокол ICMP). У нижній панелі програми автоматично відобразився шістнадцятковий дамп (hex dump) виділеного пакета.

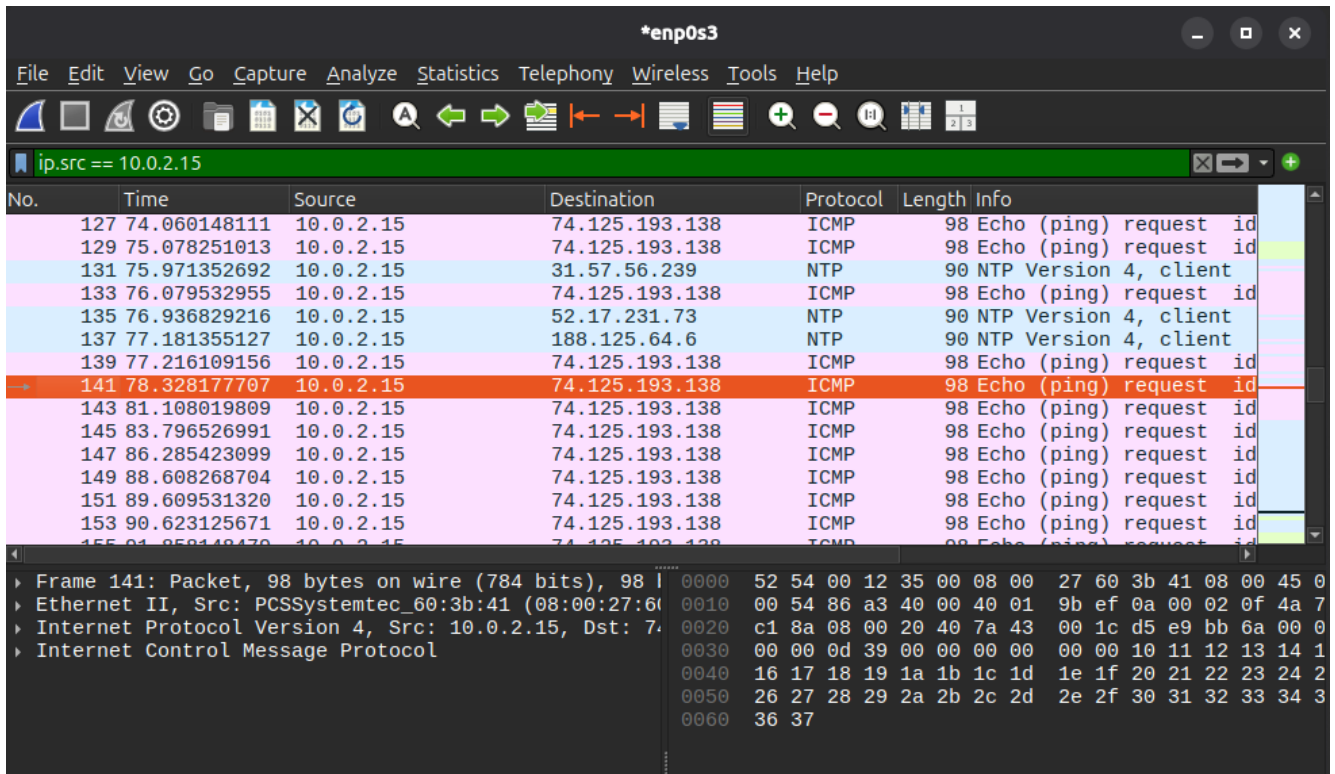


Після того, як необхідний для аналізу обсяг трафіку був успішно зібраний, процес захоплення пакетів було зупинено. Для цього було натиснуто на спеціальну червону квадратну піктограму («Stop capturing packets») на верхній панелі інструментів Wireshark.

3. Налаштування фільтрів у Wireshark



Для того, щоб прибрати зайвий трафік та залишити для аналізу лише необхідні дані, було використано вбудовану систему фільтрації програми (Display Filter). Шляхом ручного введення назви протоколу **icmp** у рядок фільтрації, програма відфільтрувала загальний потік мережевих даних, вивівши на екран виключно пакети ICMP-запитів та відповідей. Зелений фон рядка підтверджує синтаксичну правильність введеного фільтра.



Для детальнішого аналізу трафіку, згенерованого конкретним вузлом, було застосовано фільтр за IP-адресою джерела без необхідності запам'ятовування синтаксису фільтрів. Для цього у панелі деталей пакета було натиснуто правою кнопкою миші на локальну IP-адресу віртуальної машини (10.0.2.15) та в контекстному меню обрано команди **Apply as Filter -> Selected**. У результаті програма автоматично згенерувала та застосувала вираз `ip.src == 10.0.2.15` у рядку фільтрації. Після застосування цього фільтра в таблиці залишилися лише ті пакети (зокрема протоколів ICMP та NTP), ініціатором відправлення яких була локальна машина користувача.

Виконання завдання:

Згідно з рекомендаціями до виконання завдання лабораторної роботи як цільовий пристрій для експериментального дослідження протоколу ARP було використано **мобільний телефон**, підключений до тієї ж локальної мережі (Wi-Fi), що й робоча станція.

IP Address

192.168.1.3

Wi-Fi Address

6A:C0:DB:94:97:BE

Для підготовки до тестування у налаштуваннях бездротового підключення смартфона було попередньо визначено його базові мережеві параметри. Як видно з наведеного скриншота, актуальна IPv4-адреса цільового пристрою становить **192.168.1.3**, а його фізична MAC-адреса (Wi-Fi Address) — **6A:C0:DB:94:97:BE**. Отримані ідентифікаційні дані є необхідними для подальшого формування ICMP-запитів та моніторингу оновлень кешу ARP-таблиці на каналному рівні.

```
kyzym_y_k@Ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:60:3b:41 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.9/24 brd 192.168.1.255 scope global dynamic noprefixroute enp0s3
        valid_lft 86341sec preferred_lft 86341sec
    inet6 fe80::a00:27ff:fe60:3b41/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
kyzym_y_k@Ubuntu:~$
```

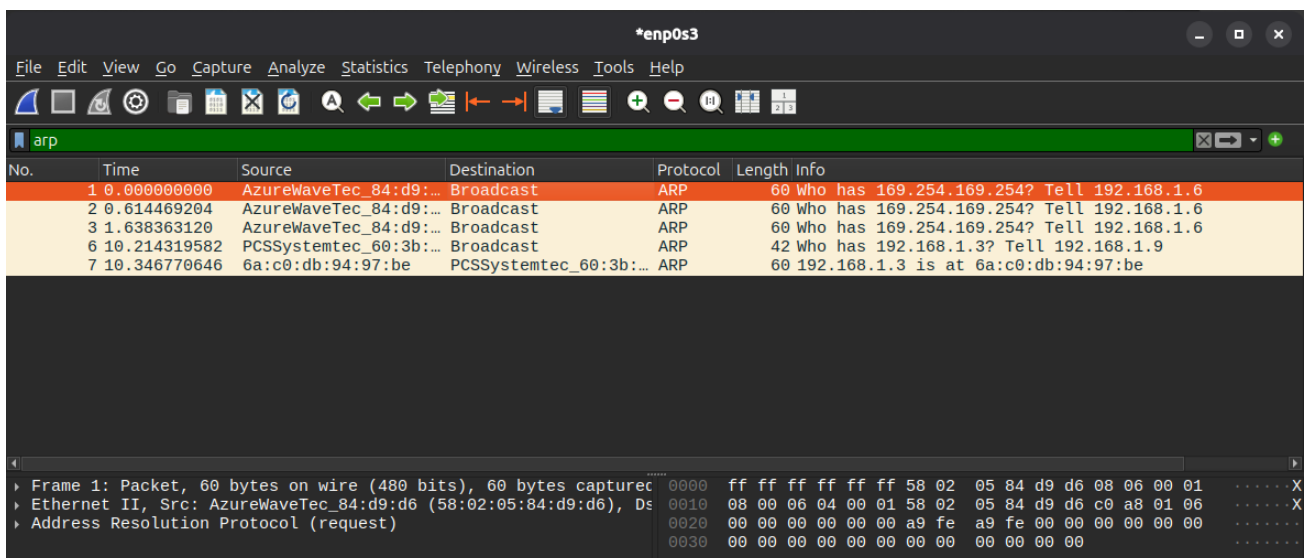
Для забезпечення прямої видимості між віртуальною машиною та цільовим пристроєм (смартфоном) на каналному рівні, тип мережевого адаптера у VirtualBox було змінено на Bridged Adapter. Перевірка мережевої конфігурації за допомогою команди **ip a** підтвердила, що локальний інтерфейс **enp0s3** успішно отримав IPv4-адресу **192.168.1.9**. Оскільки ця адреса належить до тієї ж підмережі (192.168.1.0/24), що й IP-адреса цільового смартфона, це гарантує безперешкодний обмін ARP-повідомленнями під час проведення експерименту.

1) Переконаватися в отриманні MAC-адреси по відомій IP-адресі.

Для перевірки динамічного оновлення ARP-таблиці кеш локальної машини було попередньо очищено командою **sudo arp -d 192.168.1.3**. Наступний виклик **arp -a** підтвердив успішне видалення та відсутність запису для цільової IP-адреси. Потім за допомогою утиліти **ping** було ініційовано відправлення ICMP-запитів на адресу смартфона **192.168.1.3**.

```
kyzym_y_k@Ubuntu:~$ sudo arp -d 192.168.1.3
[sudo: authenticate] Password:
No ARP entry for 192.168.1.3
kyzym_y_k@Ubuntu:~$ arp -a
www.adsl.vf (192.168.1.1) at 98:2c:c6:71:b1:78 [ether] on enp0s3
kyzym_y_k@Ubuntu:~$ ping 192.168.1.3
PING 192.168.1.3 (192.168.1.3) 56(84) bytes of data.
64 bytes from 192.168.1.3: icmp_seq=1 ttl=64 time=239 ms
64 bytes from 192.168.1.3: icmp_seq=2 ttl=64 time=18.7 ms
64 bytes from 192.168.1.3: icmp_seq=3 ttl=64 time=61.2 ms
64 bytes from 192.168.1.3: icmp_seq=4 ttl=64 time=54.9 ms
64 bytes from 192.168.1.3: icmp_seq=5 ttl=64 time=66.5 ms
^C
--- 192.168.1.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 6316ms
rtt min/avg/max/mdev = 18.664/88.012/238.844/77.254 ms
```

Оскільки фізична адреса одержувача на той момент була невідома системі, операційна система автоматично згенерувала широкомовний ARP-запит (Broadcast), що наочно зафіксовано в аналізаторі трафіку Wireshark: **Who has 192.168.1.3? Tell 192.168.1.9**. Відразу після цього цільовий пристрій успішно відправив одноадресну відповідь (arp-reply), повідомивши свою MAC-адресу: **192.168.1.3 is at 6a:c0:db:94:97:be**.



The image shows a Wireshark packet capture on the 'arp' filter. The packet list shows five packets. The first packet is an ARP request (Who has 192.168.1.3? Tell 192.168.1.9) from source 6a:c0:db:94:97:be to destination Broadcast. The fifth packet is an ARP reply (192.168.1.3 is at 6a:c0:db:94:97:be) from destination 6a:c0:db:94:97:be to source 6a:c0:db:94:97:be. The packet details pane shows the first packet's structure: Ethernet II, Src: AzureWaveTec_84:d9:d6, Dst: Broadcast, and Address Resolution Protocol (request).

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	AzureWaveTec_84:d9:d6	Broadcast	ARP	60	Who has 169.254.169.254? Tell 192.168.1.6
2	0.614469204	AzureWaveTec_84:d9:d6	Broadcast	ARP	60	Who has 169.254.169.254? Tell 192.168.1.6
3	1.638363120	AzureWaveTec_84:d9:d6	Broadcast	ARP	60	Who has 169.254.169.254? Tell 192.168.1.6
6	10.214319582	PCSSystemtec_60:3b:...	Broadcast	ARP	42	Who has 192.168.1.3? Tell 192.168.1.9
7	10.346770646	6a:c0:db:94:97:be	PCSSystemtec_60:3b:...	ARP	60	192.168.1.3 is at 6a:c0:db:94:97:be

Frame 1: Packet, 60 bytes on wire (480 bits), 60 bytes captured
Ethernet II, Src: AzureWaveTec_84:d9:d6 (58:02:05:84:d9:d6), Dst: Broadcast
Address Resolution Protocol (request)

2) Переконалися в тому, що **arp**-таблиця оновлюється при отриманні **arp-reply**.

По завершенні виконання команди **ping** повторний виклик **arp -a** у терміналі підтвердив успішне динамічне оновлення ARP-таблиці. У системі з'явився новий запис, що коректно зв'язує IP-адресу 192.168.1.3 з отриманою MAC-адресою 6a:c0:db:94:97:be на мережевому інтерфейсі **enp0s3**.

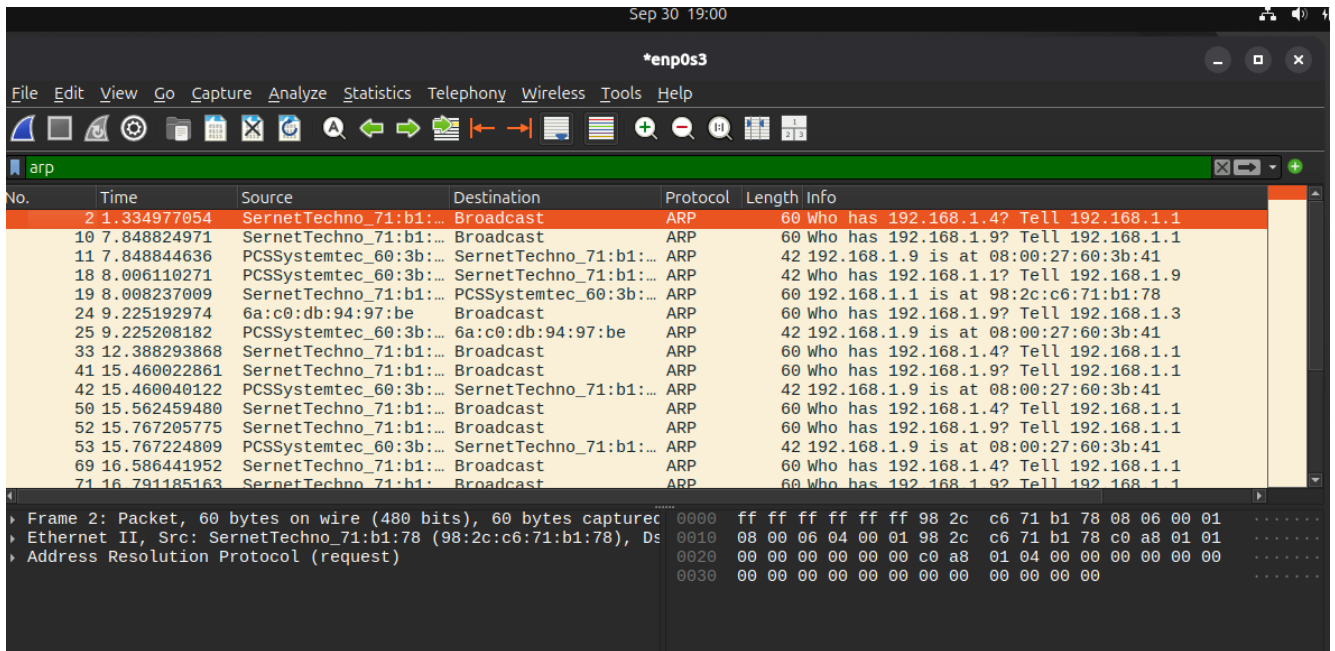
```
kyzym_y_k@Ubuntu:~$ arp -a
www.adsl.vf (192.168.1.1) at 98:2c:c6:71:b1:78 [ether] on enp0s3
? (192.168.1.3) at 6a:c0:db:94:97:be [ether] on enp0s3
```

3) Додати статичний запис у **arp**-таблицю та після цього пропінгувати інший пристрій, при цьому відстежувати трафік у **wireshark**.

Для виконання останнього завдання попередній динамічний запис було видалено командою **sudo arp -d 192.168.1.3**. Після цього MAC-адресу цільового пристрою було статично прив'язано до його IP-адреси за допомогою команди **sudo arp -s 192.168.1.3 6a:c0:db:94:97:be**. Перевірка командою **arp -a** підтвердила успішне виконання операції: біля відповідного запису з'явився маркер **PERM** (permanent), який вказує на те, що запис є постійним і не підлягає автоматичному оновленню.

```
kyzym_y_k@Ubuntu:~$ sudo arp -d 192.168.1.3
[sudo: authenticate] Password:
kyzym_y_k@Ubuntu:~$ sudo arp -s 192.168.1.3 6a:c0:db:94:97:be
kyzym_y_k@Ubuntu:~$ arp -a
www.adsl.vf (192.168.1.1) at 98:2c:c6:71:b1:78 [ether] on enp0s3
? (192.168.1.3) at 6a:c0:db:94:97:be [ether] PERM on enp0s3
kyzym_y_k@Ubuntu:~$ ping 192.168.1.3
PING 192.168.1.3 (192.168.1.3) 56(84) bytes of data.
64 bytes from 192.168.1.3: icmp_seq=5 ttl=64 time=5.12 ms
64 bytes from 192.168.1.3: icmp_seq=6 ttl=64 time=18.0 ms
64 bytes from 192.168.1.3: icmp_seq=7 ttl=64 time=37.1 ms
64 bytes from 192.168.1.3: icmp_seq=8 ttl=64 time=39.1 ms
64 bytes from 192.168.1.3: icmp_seq=9 ttl=64 time=42.5 ms
64 bytes from 192.168.1.3: icmp_seq=10 ttl=64 time=18.6 ms
64 bytes from 192.168.1.3: icmp_seq=11 ttl=64 time=29.0 ms
64 bytes from 192.168.1.3: icmp_seq=12 ttl=64 time=47.4 ms
^C
--- 192.168.1.3 ping statistics ---
12 packets transmitted, 8 received, 33.3333% packet loss, time 12959ms
rtt min/avg/max/mdev = 5.115/29.596/47.388/13.629 ms
```

Під час подальшого відправлення ICMP-запитів (команда **ping 192.168.1.3**) паралельно проводився моніторинг трафіку у програмі **Wireshark** із застосованим фільтром **arp**. Експеримент успішно підтвердив, що локальна машина не генерувала жодних нових широкомовних ARP-запитів для пошуку MAC-адреси цільового вузла, оскільки необхідні дані вже були заздалегідь відомі системі з локальної статичної ARP-таблиці.



Тестова документація

Поле	Значення
Ідентифікатор	ARP-ТС-01
Назва	Додавання статичного запису в ARP-таблицю та аналіз трафіку
Передумови	Дві машини (робоча станція 192.168.1.9 та смартфон 192.168.1.3) знаходяться в одній локальній мережі; кеш ARP для цільової адреси на робочій станції очищено; запущено захоплення пакетів у Wireshark на локальному інтерфейсі з фільтром arp.
Кроки	1) додати запис через <code>sudo arp -s 192.168.1.3 6a:c0:db:94:97:be</code> ; 2) перевірити <code>arp -a</code> ;

	3) виконати ping 192.168.1.3; 4) проаналізувати захоплений трафік у Wireshark.
Очікувано	У виводі команди arp -а наявний запис для цільової IP-адреси з маркером PERM; у програмі Wireshark відсутні вихідні ARP-запити (Broadcast: Who has 192.168.1.3?) від локальної машини.
Фактично	Запис для 192.168.1.3 містив MAC-адресу 6a:c0:db:94:97:be і стан PERM; перед ICMP-пакетами відсутній новий ARP Request клієнта для цієї адреси
Результат	PASS. Очікуваний і фактичний результати збігаються
Післяумова	Статичний запис видалено командою sudo arp -d 192.168.1.3.

Висновок: Під час виконання лабораторної роботи було успішно налаштовано мережне оточення та досліджено механізми роботи протоколу ARP на прикладі взаємодії віртуальної машини та мобільного пристрою. Опановано інструментарій Wireshark для захоплення і детального аналізу трафіку на канальному рівні. У межах індивідуального завдання експериментально доведено, що додавання статичного запису в ARP-таблицю запобігає генерації нових ширококомовних запитів під час звернення до відомого вузла. Отримані практичні результати успішно формалізовано у вигляді тестової документації (Test-case) згідно з принципами QA Embedded Testing. Це дозволило закріпити навички, необхідні для подальшого тестування мережних вбудованих систем.

Репозиторій: [посилання](#).